Distilling `phi3.5` into Pythia-70M and deploying it on a local cloud

CS3081 Cloud Computing, mini project report

Name: **Sameer Patel** Roll number: **230101040** Repository: github.com/urffsamhunt/cloud-mini-project

How to read this report. Numbers marked **Measured** come from files in `results/` (the file is named next to them). Anything marked **Assumption** is a choice or an estimate, not a measurement. The submitted model is **v2** (GitHub Release `model-v2`); v1 is the first model and the baseline for the rolling-update test. Everything ran on one laptop (Fedora 43, CPU only for training and serving).

Architecture and main decisions

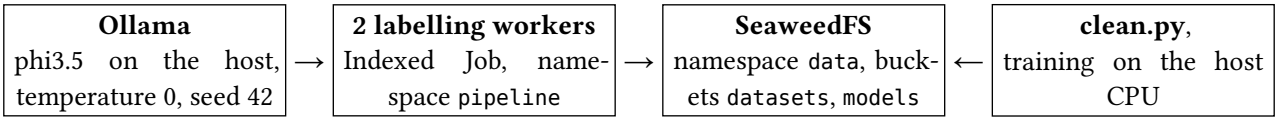


Figure 1: Data path (Tasks 1 and 2). Training and cleaning read and write the same object store.

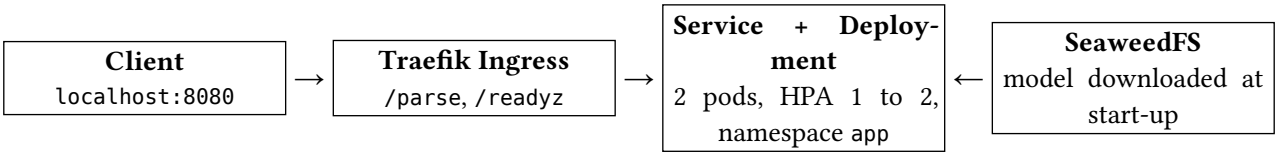


Figure 2: Serving path (Tasks 3 and 4). The image holds no weights; each pod downloads and checksums its model.

Decisions. (1) Stack: k3d for Kubernetes, SeaweedFS for S3 storage, Ollama for the teacher; all open source and local, as the handout requires. (2) Ollama runs on the host as a systemd service (`OLLAMA_HOST=0.0.0.0`) and the worker pods reach it at `host.k3d.internal:11434`; this was allowed by the handout and avoids a second 2 GB copy of the model in the cluster. (3) Training runs on the host CPU in a virtual environment (the handout asks for CPU); only the result goes to object storage. (4) Credentials are generated at deploy time (`scripts/create_secrets.sh`) and exist only as Kubernetes Secrets. (5) One command rebuilds everything: `make up`. Every step that was run, and whether it was tested, is in `docs/SETUP_LOG.md`.

Task 1: Object storage and parallel teacher labelling

Setup. `k3d cluster create --config cluster/k3d.yaml` (one server node, host port 8080 mapped to the load balancer). SeaweedFS (`chrislusf/seaweedfs:4.51`, S3 on port 8333) runs in namespace `data` on a 4 GiB volume; buckets `datasets` and `models` exist and `datasets` holds `commands_pool.jsonl` (`scripts/init_buckets.py`). The teacher prompt is `prompts/teacher_prompt.txt` (task description, the eight intents, the three slot rules and 12 worked examples, none taken from the validation set). The requested options are `temperature 0, seed 42`. I added `num_predict 128` and `stop ["\n"]`: without the stop, the replies in my first test often carried an explanation after the JSON, and only 6 of 10 were valid JSON; with it, 20 of 20 test replies (10 commands, each twice) were valid and identical across the two runs (`scripts/test_teacher.py`). This is a deviation from the bare settings in the handout and is stated here on purpose.

Counts (**Measured**, `results/clean_report.json`). The rejection rules are applied in the order (a) to (d); a record is counted under the first rule it breaks; duplicates are removed afterwards among the survivors.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	(e) duplicates	Kept	Check
3000	4	1	2	40	50	2903	4+1+2+40+50+2903 = 3000

`train.jsonl` has SHA-256 `dfcdc41d…6671f` (`datasets/v1/train.sha256`); running `clean.py` again on the raw shards, both from SeaweedFS and from the copy committed in `data/teacher/`, gives the same checksum. The two workers took 21 minutes in total (07:13 to 07:34 UTC) on the CPU.

Sharding (half a page). The worker reads `SHARD_INDEX` and `SHARD_COUNT` and keeps the commands whose line number n (counting from 0 over all 3,000 lines) satisfies $n \bmod \text{SHARD_COUNT} = \text{SHARD_INDEX}$. With two workers, worker 0 gets the even lines and worker 1 the odd lines. Each line belongs to exactly one remainder class, so the shards cannot overlap or leave gaps: **Measured** shard 0 has 1,500 ids, shard 1 has 1,500 ids, their intersection is empty and their union is all 3,000 ids (`data/teacher/`). Modulo sharding rather than splitting the file in halves keeps the shards balanced if the file is ordered (for example by intent or length) and needs

no coordination between workers. The workers run as one Kubernetes Indexed Job (`k8s/teacher-job.yaml`), which sets `SHARD_INDEX` from the pod's completion index (`batch.kubernetes.io/job-completion-index`) and runs both pods at once in namespace `pipeline`. Each worker writes its reply line immediately and can resume from its local file after a restart, then uploads `teacher/shard-N.jsonl` to its own object. The Ollama server (`OLLAMA_NUM_PARALLEL=1`) is the shared bottleneck: two workers overlap network, JSON and queueing time but the generation itself is serialised, so the speed-up from the second worker is smaller than 2×. **Assumption** I did not measure a single-worker run, so I cannot give the speed-up.

Task 2: Training the student (sequence-level distillation)

Training. `make train` reads only `training/config.yaml`, verifies the dataset checksum, and fine-tunes `EleutherAI/pythia-70m` on the first 1,500 records of `train.jsonl` on the CPU. An example is `Command: <text>\nJSON: <teacher reply>`; the loss is computed on the reply tokens only (the prompt tokens are masked). `make train SUBSET=50` trains on 50 records and writes only to `out/subset-test/` (**Measured**: it ran to the end in 19 s). The checkpoint is stored as float16, so it must be loaded as float32 for training (float16 backward on the CPU was roughly 1,000 times slower; the first run did not finish 50 examples in 10 minutes).

Seed	Learning rate	Batch size	Epochs v1 / v2	Max length	Warm-up	Optimiser	Libraries
42	2e-4	16	5 / 6	128	10 % linear	AdamW, wd 0	torch 2.15.0+cpu, transformers 5.22.1, numpy 2.5.4

Training loss per epoch (**Measured**, `training/loss.csv`, `training/loss_v2.csv`; token-weighted mean over the epoch):

Epoch	1	2	3	4	5	6
v1	0.498	0.253	0.152	0.076	0.034	–
v2	0.485	0.295	0.179	0.106	0.054	0.024

Artefacts. Both models are in SeaweedFS under `models/v1/` and `models/v2/` (model, tokenizer, `version.json`, `files.sha256`, `model.sha256`) and are GitHub Release assets `model-v1` and `model-v2`. `model.sha256` is the SHA-256 of the deterministic archive `model-<v>.tar`: v1 `e7b7b823…85963`, v2 `a775ecc5…eba1`; **Measured** the Release asset digests reported by GitHub are identical (`gh release view`). v2 differs from v1 only by one more epoch (6 instead of 5).

Three models on the validation set (**Measured**, `tools/score.py`, 100 commands, percent; raw outputs in `results/*_predictions.jsonl`, scores in `results/*_score.json`):

Model	JSON validity	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (`teacher_validation.json`)	99	85	80	87	87	54
Untrained Pythia-70M	0	0	0	0	0	0
Student v1	99	85	82	84	86	49
Student v2 (submitted)	**100**	**85**	**79**	**93**	**87**	**51**

What changed. The untrained model never produces the format (it continues the text), so every score is 0. After training on 1,500 teacher replies the student produces valid JSON in 99 to 100 % of the cases and reaches 91 to 94 % of the teacher's full-frame score (49/54 and 51/54). Intent accuracy equals the teacher's (85). Differences between the student versions are small: one command is one point, and v2 has 100 % validity, a better time score (93 against 84) and a worse date score (79 against 82), so v2 is not better on every measure. The student gives exactly the teacher's text on 62 (v1) and 65 (v2) of 100 commands. The teacher is wrong on 46 commands; on 19 (v1) and 22 (v2) of these the student gives the same wrong answer, which shows what was copied.

Sequence-level and logit-based distillation (one page or less). In **sequence-level** distillation (Kim and Rush, 2016) the teacher is run once on every input and produces a complete output sequence, here the JSON text it generates with temperature 0, which is its most likely sequence. The student is then trained as an ordinary language model on these texts with the cross-entropy loss: for each position it must give high probability to the next token of the teacher's text. The only thing taken from the teacher is the **text** it generated; the student sees one correct answer per input and no information about what else the teacher considered. In **logit-based** distillation the student is trained on the teacher's **output distribution**: at each position both models produce a probability distribution over the vocabulary, and the loss is the KL divergence between the teacher's distribution (often softened with a temperature) and the student's. This carries much more information per token, because the

teacher's probabilities for the wrong tokens show which alternatives are close, and it can be used with unlabelled text. It needs the teacher's logits at every position and the same, or a mapped, vocabulary on both sides.

Logit-based distillation is not possible in this project for three reasons. (1) The teacher is served through Ollama, and what this project stores are the **generated replies** (`teacher_raw`); no distribution over the vocabulary is returned or saved, and the handout asks to train on the stored replies only. (2) The two models do not share a vocabulary: Phi-3.5 uses a SentencePiece vocabulary of about 32 thousand tokens and Pythia the GPT-NeoX BPE vocabulary of about 50 thousand tokens, so the probabilities at one position are over different token sets and cannot be compared without an alignment. (3) Getting logits would mean running the 3.8-billion-parameter teacher in a deep-learning framework next to the student on a CPU laptop, whereas Ollama's quantised build runs on the laptop. Therefore only the text of the reply can be used.

One limitation of what I did. The student can only be as good as its teacher and it copies the teacher's systematic mistakes: **Measured** the teacher's full-frame score on the validation set is 54 %, and the student reproduced the teacher's wrong answer on 19 to 22 of the 46 commands that the teacher got wrong. The cleaning rules only remove replies that are malformed or invent slot text (97 of 3,000 records); they cannot remove replies that are well-formed but wrong, for example a wrong date or intent. A second weakness is the evaluation: it uses 100 commands, so a result has a statistical uncertainty of about ±5 points, and the final training loss (0.034 for v1) is close to zero, which suggests that the model has partly memorised the 1,500 examples (**Assumption** I did not measure the loss on held-out data).

Task 3: The inference service in a container

`service/app.py` (FastAPI, port 8000) has `POST /parse`, `GET /healthz` and `GET /readyz`. Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT` and `MAX_NEW_TOKENS` (plus `FAULT_INJECT`); credentials come from the Secret `s3-creds` as `AWS_*` environment variables. On start-up a background thread downloads the files from `MODEL_URI`, verifies them against `files.sha256`, and loads the model; `/readyz` returns 503 until then. If the download or checksum fails, the process exits so Kubernetes restarts the pod. Every output is checked with `jsonschema` against `schema.json`; an invalid output returns HTTP 500 with the body `{"error": "invalid_model_output"}`, never broken JSON with 200. Responses carry `X-Model-Version` and `X-Served-By` (the pod name). With `FAULT_INJECT=1` the output is replaced by `not json` before the check. The torch thread count follows the container's CPU limit (read from the cgroup).

Image. `service/Dockerfile` starts from `python:3.12.16-slim`, installs the CPU-only PyTorch wheel (the default wheels carry GPU libraries) and every dependency pinned to an exact version (`service/requirements.txt`), runs as user 10001 and contains no weights. **Measured** Size 1,309,365,686 bytes (1.31 GB, limit 2 GB), 0 weight files inside, built with one command (`make service-image`).

Checks (**Measured**, `results/service_api.txt`, `results/service_limits.txt`, run with model v2): `/healthz` 200; `/readyz` 503 while loading, 200 about 4 s later; `/parse` returns the four-key object with both headers; with `FAULT_INJECT=1` the response is HTTP 500 with exactly `{"error": "invalid_model_output"}`; an empty text gives 422. **Limits test** (`docker run --cpus=1 --memory=1g --memory-swap=1g`, 50 requests one after another): 50 × HTTP 200, p50 430 ms, p95 519 ms, **peak memory 826,273,792 bytes (788 MiB, cgroup `memory.peak`)**, no out-of-memory kill. `docker stats` showed 364 MiB in use after the run; the peak includes file-cache pages from reading the model, which the kernel can reclaim. The headroom to the 1 GiB limit is about 236 MiB. (An earlier run with model v1 gave 49 × 200 and 1 × 500, the schema check catching one invalid output, and a peak of 811,036,672 bytes.)

Task 4: Kubernetes deployment, scaling and reliability

Deployment (`k8s/app.yaml`, namespace `app`): a Deployment with 2 replicas (CPU request 500 m and limit 1 CPU; memory request 768 Mi and limit 1 Gi), a Service on port 80 and a Traefik Ingress that sends `/parse` (and `/readyz`) to it, a startup and a liveness probe on `/healthz`, a readiness probe on `/readyz`, `maxUnavailable: 0` and a `preStop` delay of 8 s. The HPA has minimum 1, maximum 2 and a 70 % CPU target. `curl localhost:8080/parse` answers through the Ingress. The pods pull `parser-service:1`, which is imported into k3d (`k3d image import`). The default model is v2 (`MODEL_URI=s3://models/v2/`). All tests below are produced by `scripts/task4_evidence.sh`, `scripts/task4_scaling.sh` and `scripts/loadtest.sh`; the load always comes from Locust (4 users) with longer run times where something is changed during the run.

Test	Result (Measured)	File in `results/`

Service has no ready endpoints while the model loads	After scaling to 1 pod, the pod's IP is listed under `notReady` and `GET /readyz` through the Ingress returns 503 for about 6 s; when the pod becomes ready the IP moves to `ready` and `/readyz` returns 200.	`task4_endpoints_loading.txt`
HPA 1 → 2 under load, and back	Start with 1 replica. About 10 s after the load starts the HPA reports 125 % CPU and desires 2 replicas; 2 pods are ready at 21:37:34 (197 % of the request). After the load stops (21:41) the HPA keeps 2 replicas for its default 5-minute stabilisation window, desires 1 at 21:46:17 and is down to 1 pod at 21:46:38.	`task4_scaling.txt`, `task4_scaling_stats.csv`
Rolling update v1 → v2, then `kubectl rollout undo`, under load (4 min)	Update completed in 15 s; after the rollback `X-Model-Version` shows **v1** again (`rollout undo`, 30 s until the pods were ready). 1,180 requests, 10 failures (**0.85 %**), all HTTP 500 `invalid_model_output`, none 502/503 or connection errors. These are model v1′s known error on one validation command, which comes back once per 100 requests while v1 serves. **Same test with that command removed (99 commands): 707 requests, 0 failures.** So the rollout itself lost no request; the 0.85 % is model v1′s own error rate, and it is above the 0.5 % limit if counted as a whole.	`task4_rollout.txt`, `task4_rollout_without_V049.txt`
Recovery: delete one pod under load	Two ready replicas again after 10.4 s (`readyReplicas = 2`); 334 requests, 0 failures (0 %).	`task4_recovery.txt`
Cold start: delete all pods, 3 runs	Creation → Ready: 7.0 s, 7.0 s, 6.0 s for the slower of the 2 pods, **median 7.0 s** (Kubernetes timestamps have 1 s resolution). The logs show for every pod `downloaded 7 files (285.3 MB) from s3://models/v2/`, `checksums verified`, `model v2 ready after 4.1 s`.	`task4_coldstart.txt`, `task4_coldstart_logs.txt`
Load balancing, exactly 2 Ready replicas, standard command	504 successful requests: **252 and 252** (50 % each, limit 30 %), all served by v2.	`loadtest_headers.json`

Limitation of the local cluster. The cluster is a **single node on one laptop**: Locust, Traefik, both replicas, SeaweedFS and the host share the same CPU cores, so replicas do not add capacity the way they would on separate machines (the throughput of two replicas was 4.2 requests per second while each pod already used about 99 % of its CPU limit), and none of these tests shows fault tolerance across machines: if the laptop or the node fails, everything fails. SeaweedFS is also a single pod on one local volume, so a new pod cannot start its model download while it is down. **Assumption** A multi-node cluster would behave differently; I could not test it. Notes on the measurements: the HPA was removed while the recovery, rollout, cold-start and load-balancing tests ran (an idle HPA scales back to 1 replica before the load begins) and restored afterwards; the p95 latency of the clean rollout run (3.7 s) is high because three pods and a model download share the CPU during the update.

Task 5: Subnetting and network access control

Subnets for 10.20.0.0/16, largest first, each directly after the previous one, with the smallest prefix that fits (usable hosts = $2^{32-\text{prefix}} - 2$; working in `network/subnets.md`):

Subnet	Hosts needed	Prefix	Network	Usable range	Broadcast	Usable hosts
app	1,000	/22	10.20.0.0	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL for the public subnet (`network/nacl.json`). Inbound: 100 deny all from 198.51.100.0/24; 110 allow TCP 80 and 120 allow TCP 443 from 0.0.0.0/0; 130 allow TCP 22 from 10.20.5.128/27 (management); 140 allow TCP 1024–65535 from 10.20.0.0/22 (replies from the app subnet). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Everything else falls to the default deny. **Measured** `tools/nacl_sim.py --check` decides all 6 sample packets correctly (100 %) and all 23 extra packets that I wrote (blocked range on ports 80, 443, 22 and a high port; SSH from inside and outside the management range; replies; UDP; ports just below 1024) as I expected (23 of 23; these expectations are my reading of the task, not the marker's). A limit of the rules: the reply rule (TCP 1024–65535) also lets through other high-port traffic, for example outbound port 8000 to addresses outside the app subnet, because a stateless NACL sees only the destination port and cannot tell a reply from a new connection.

Why replies need their own rules, and why the order matters. A NACL is stateless: it checks every packet on its own and keeps no record of connections. A request to port 443 is checked by the inbound rules, but its reply travels the other way, from port 443 to the client's ephemeral port, and is checked by the outbound rules

as unrelated traffic. Without an outbound rule for TCP 1024–65535 the replies would meet the default deny and every connection would hang although the request was accepted. The same holds for the load balancer's request to the app on port 8000, whose answer returns to the load balancer's ephemeral port and needs inbound rule 140. A stateful security group allows the reply automatically. Rules are evaluated from the lowest number upward and the first match decides, so the deny for 198.51.100.0/24 works only if its number is lower than the allow rules it must override: with rule 100 a packet from 198.51.100.9 to port 443 matches the deny first and is dropped; if the deny were numbered 150, the same packet would match allow rule 120 first and the deny would never be reached.

Task 6: Reproducibility, testing and CI/CD

One command from a fresh clone. `git clone … && cd … && make up` builds and deploys everything: it checks the tools, creates the k3d cluster, namespaces, Secrets, SeaweedFS and buckets, downloads the two models from the GitHub Releases (verified against `release/model-<v>.sha256`) and uploads them, builds the worker and service images, imports them and applies `k8s/app.yaml`. `make down` deletes the cluster. **Measured** I ran `make down`, cloned the GitHub repository into a new directory and ran `make up`: both archives checksum OK, two pods ready and `POST localhost:8080/parse` returned HTTP 200; the second run (the first one stalled, see below) took 2 min 34 s. The stall was a model download that hung without a timeout (181 MB of 285 MB, 12 minutes); `scripts/fetch_models.py` now has a timeout, retries and resume.

CI (`.github/workflows/ci.yml`; jobs in this order): `secret-scan` (gitleaks, next to lint), `lint` (ruff), `unit-tests` (13 pytest tests: the cleaning rules, schema validation, probes before the model is loaded, the prompt format, the `score.py` quality gate on bad and perfect predictions, and the NACL file), `evaluate` (downloads `model-v2.tar` from the GitHub Release, checks it against the checksum committed in the repository, runs `training/predict.py` and `tools/score.py` on `validation.jsonl`) and `quality-gate` (`score.py --min-validity 95`). The model to evaluate is a workflow input.

Run	Result	Link
Release model-v2 (the default), push	all five jobs green, gitleaks found nothing; validity 100, intent 85, date 79, time 93, full frame 51 (same as locally)	actions/runs/42503863202
Release model-v1, push	all five jobs green, gitleaks found nothing; validity 99, intent 85, full frame 49 (same as locally)	actions/runs/42459095876
Untrained Pythia-70M (manual run, `model_source = untrained`)	**fails at the quality gate**; the earlier jobs are green	actions/runs/42459268412

Measured The runs are listed in `results/ci_runs.md`. The workflow default is `model-v2` (the deployed model); the v1 run was made before the default was changed. Secret scanning: gitleaks also reported no leak when run locally over the git history, and credentials exist only as Kubernetes Secrets. `AI_USE.md` states which AI tools were used (Claude Code, which wrote most of the code and scripts at my request, and the phi3.5 teacher), what they were used for, how the output was checked (scripts were run, counts and checksums compared with the saved logs) and which checks were not done.

Task 7: Performance, cost and review

Load test (**Measured** standard command, 4 users, 2 minutes, `localhost:8080`, model v2, two ready replicas; `results/loadtest_*.csv`, `loadtest_headers.json`, `loadtest_console.txt`, `loadtest_top.txt`):

Requests	Failures	Error rate	p50	p95	Throughput	Cold start (Task 4)
500	0	0 %	940 ms	1,500 ms	4.21 req/s	median 7.0 s

CPU and memory from `kubectl top pods` (every 10 s during the run): CPU 986–993 m per pod (limit 1,000 m, request 500 m), memory 383–566 MiB per pod (limit 1 GiB). Both thresholds of the handout are met (error rate at most 1 %, p95 at most 5,000 ms). For comparison, the same test with model v1 (`loadtest_v1_*`, started with one replica that the HPA scaled to two) had 432 requests, 4 failures (0.93 %), p50 1,300 ms, p95 1,600 ms and 3.63 req/s; the 4 failures are the one command that v1 answers with broken JSON, which is why v2 is the deployed model (v1's expected error rate is 1 %, too close to the limit).

Cost (**Assumption** rates from `data/rate_card.json`: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; billed on the limits; an estimate, nothing is charged). (1) One replica: 1 vCPU × 3.50 + 1 GiB × 0.45 = 3.95 INR per hour. (2) Two replicas ran during the test (**Measured**): 2 × 3.95 = 7.90 INR per hour. (3) Requests served per hour = 500 successful requests in 120 s × 30 = 15,000. (4) Cost per 1,000 requests = 7.90 / 15,000 × 1000 = **0.527 INR**.

SLO. Target: 99 % of `POST /parse` requests return HTTP 200 with a valid four-key object within 30 s. Window: one load-test run (120 s). Error budget = 1 % of 500 requests = **5 requests**; used: **0 (0 %)**. For the v1 run: budget 4.32 requests (4 whole requests), used 4, that is 92.6 % of the budget.

Review (full table with the same numbers in `results/task7_analysis.md`):

Component	Evidence (measured)	Limitation found	Improvement
Data and storage	3,000 teacher records, 2,903 kept; 285 MB per model on a 4 GiB volume	One SeaweedFS pod, no replication; 40 of the 97 rejections are slots the teacher invented	Replicate or back up the volume; stricter slot rule in the prompt
Model quality	v2 validity 100 %, full frame 51 % against the teacher's 54 % (100 commands); final loss 0.024	100 commands only (±5 points); student copied 22 of the teacher's 46 errors; loss near 0 hints at overfitting	More data and a held-out loss; constrained JSON decoding
Latency and throughput	p50 940 ms, p95 1,500 ms, 4.21 req/s; alone 430 ms	CPU-bound; 4 users on 2 CPUs queue; closed-loop test	Smaller `MAX_NEW_TOKENS`, batching, int8 or ONNX
Resource utilisation	CPU 986–993 m of 1,000 m; memory 383–566 MiB of 1 GiB	CPU request (500 m) is half of the real use; memory grew from 383 to 566 MiB on one pod	CPU request about 900 m; longer run to check the memory growth
Reliability and recovery	Pod deletion: 10.4 s, 0 of 334 failed; rollout without V049: 0 of 707 failed; cold start median 7.0 s	One node; SeaweedFS is a single point of failure	Multi-node cluster; model cache or init-container retry; replicated storage
Security and cost	gitleaks: 0 findings; Secrets only in Kubernetes; 0.527 INR per 1,000 requests	One shared full-access S3 key in three namespaces; no authentication or TLS on `/parse`; idle replicas cost 7.90 INR per hour	Read-only keys, TLS and an API key, encrypted Secrets; scale to 1 replica when idle

Limitations and improvements (summary)

The student cannot be better than its teacher and copies its mistakes (full frame 51 % against 54 %); the evaluation set is small; the cluster is one node with a single storage pod; the NACL reply rule is wider than strictly needed; the model is served at about 4 requests per second on two CPUs. Not tested: `make train` giving identical weights on a second run (the seed is fixed and deterministic algorithms are requested, but I did not rerun it), the retry path of the model download when a connection fails outright, and the Task 6 workflow with the new default (see `results/ci_runs.md`). Setup steps and what was run or not run are logged in `docs/SETUP_LOG.md`.